



**POLYTECHNIQUE
MONTRÉAL**

UNIVERSITÉ
D'INGÉNIERIE

INF3500 - Conception et réalisation de systèmes numériques

Laboratoire 2 Feu de circulation

Florian DELHON

Hiver 2026

Introduction

À la fin de ce laboratoire, vous devrez être capable de :

- Concevoir un module VHDL séquentiel à partir de ses spécifications, en passant par la conception d'un diagramme d'états
- Concevoir un banc d'essai VHDL pour valider ce module combinatoire
 - Utiliser les différents types VHDL
 - Générer des stimuli pour un circuit séquentiel
 - Utiliser les assertions (énoncé `assert/report`)
- Effectuer la validation matérielle sur FPGA, après synthèse, implémentation et programmation
 - Utiliser des interfaces comme des boutons et des LEDs
 - Corriger les phénomènes de rebond des boutons et commutateurs, et traiter les appuis simultanés

Ce laboratoire s'appuie sur et complète le matériel suivant :

1. Concepts couverts dans les **Laboratoires 0 et 1**
2. Matière du cours « **Thème 03 - Circuits combinatoires** »
3. Matière du cours « **Thème 04 - Circuits séquentiels** »

Liste des fichiers

Fichier	Contenu
src/feu.vhd	Module implémentant votre machine à états
src/monopulseur.vhd	Module monopulseur (<i>debouncer</i> en anglais) qui sert à supprimer les effets de rebond des boutons
src/top.vhd	<i>Top level</i> du projet
src/feu_tb.vhd	Banc d'essai du module feu
xdc/PYNQ-Z2-v1.0.xdc	Fichier de contraintes pour la PYNQ-Z2
synth-impl/create_zynq_wrapper.tcl synth-impl/zynq.tcl	Scripts <i>wrappant</i> le <i>top level</i> dans un design contenant les connexions nécessaires avec le processeur ARM

Séance de laboratoire

Une séance de 3 heures sera dédiée à la réalisation de ce laboratoire : le **12 février 2026 pour le groupe 2**, et le **19 février 2026 pour le groupe 1**. Elle aura lieu à **8h30**, dans la salle **L-3714**.

Remise de code

Une remise de votre code est demandée sur votre dépôt **GitHub**. Pour ce faire, un *push* final contenant l'ensemble des fichiers nécessaires doit être fait avant le **25 février 2026 à 22h, pour les deux groupes**. Vous êtes libre de faire les *push* intermédiaires souhaités avant cette date. Il vous est demandé de **ne pas modifier les noms des fichiers/entités/architectures, ni la liste des generic/port**.

Entrevue/démonstration

Votre compréhension des travaux effectués dans ce laboratoire sera évaluée lors d'**une entrevue en présentiel**. Elle se tiendra au cours de la séance dédiée au laboratoire 3 : le **26 février 2026 pour le groupe 2**, et le **12 mars 2026 pour le groupe 1**. Bien que l'entrevue se déroule en binôme, il sera demandé que **les deux élèves répondent de manière équilibrée** aux questions posées. Un élève n'ayant pas démontré une compréhension suffisante lors de l'entrevue pourra être pénalisé sur sa note individuelle.

Barème

Ce laboratoire sera noté sur 20, et comptera pour 4% de votre note finale.

Critères	Points
Remise de code	8
Partie 3	
• Votre module <code>feu.vhd</code> fonctionnel	3
• Votre banc de test <code>feu_tb.vhd</code> complet	3
Partie 4	
• Votre <i>bitstream</i> <code>top.bit</code> fonctionnel	1
Partie 5 [Défi]	
• Votre clignotement du feu jaune (<code>feu.vhd</code> , <code>feu_tb.vhd</code> et <code>top.bit</code> modifiés)	1
Entrevue/démonstration	12
Partie 1	
• Votre résolution du problème du code de départ	1
Partie 2	
• Votre diagramme d'états	1
Partie 3	
• Votre conception de l'architecture <code>feu.vhd</code>	3
• Votre démarche de validation comportementale <code>feu_tb.vhd</code>	3
Partie 4	
• Votre démonstration sur la PYNQ-Z2 du feu de circulation	2
Partie 5 [Défi]	
• Votre conception/validation du clignotement du feu jaune	2
Total	20

! Évaluation

Pour vous aider à identifier les points évalués, vous pourrez vous fier aux encadrés rouges comme celui-ci. Veillez à bien les lire afin d'obtenir la note maximale.

Partie 0 : Préparation de l'environnement de travail

Initialisation du projet Vivado

Grâce aux sources fournies dans votre dépôt de départ, **initialisez un nouveau projet Vivado**. La procédure est détaillée dans la **Note 1**.

Partie 1 : Machine à états basique

Contexte

Votre objectif dans ce laboratoire est de concevoir une machine à états de Moore pour simuler le fonctionnement d'un feu de circulation. À partir d'une spécification donnée, vous devrez :

- concevoir une machine à états et donner le diagramme d'états correspondant,
- l'implémenter en VHDL,
- et vérifier son fonctionnement en la programmant sur votre PYNQ.

Dans cette **Partie 1**, on vous présente le module déjà implémenté qui vous servira de base dans la suite du laboratoire. Dans la **Partie 2**, vous devrez concevoir une machine à états qui respecte une spécification donnée, que vous implémenterez et simulerez dans la **Partie 3**. La PYNQ sera programmée pour vérifier le bon fonctionnement de votre machine dans la **Partie 4**, et le défi de la **Partie 5** consistera à implémenter une fonctionnalité supplémentaire à votre machine.

Exemple : Des couleurs qui défilent !

Le fichier `feu.vhd` contient un exemple de machine à états basique, implémentant le diagramme d'états présenté dans la Figure 1 (page suivante). Cette machine commence dans l'état `e_0`, dans lequel les deux LEDs sont éteintes (état initial). Ensuite, chaque appui du Bouton 0 (`BTN0`, le plus à droite) fait passer la machine à l'état suivant, ce qui a pour effet de faire défiler les couleurs vert, blanc et rouge sur les deux LEDs RGB. L'état `e_3` reboucle sur l'état `e_1`. Le seul moyen d'atteindre l'état `e_0` est de réinitialiser la machine en utilisant le `reset`, relié au Bouton 3 (`BTN3`, le plus à gauche).

Prenez le temps de bien comprendre l'implémentation fournie. En particulier, les quatre états de cette machine sont déclarés dans l'architecture du module `feu` par :

```
type etat_type is (e_1, e_2, e_3, e_0);
```

L'état du système est alors porté dans le module par le signal `etat` :

```
signal etat : etat_type := e_0;
```

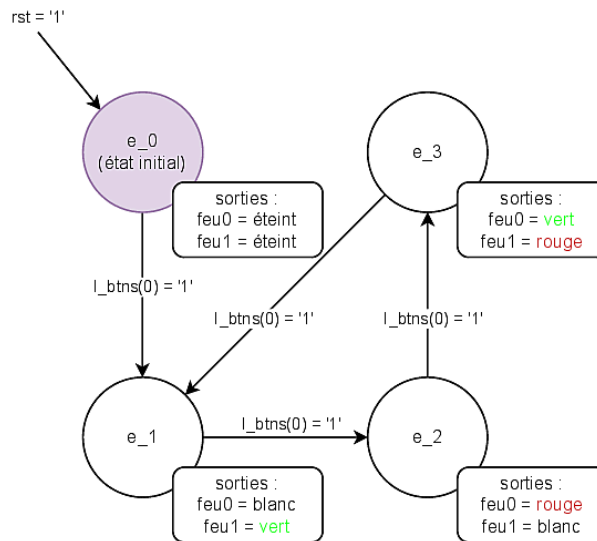


Fig. 1. – Machine à états basique

i Types abstraits en VHDL

Le mot-clé **type** déclare un type abstrait, un objet VHDL qui permet de manipuler des valeurs sans se soucier de leur valeur numérique (semblable aux **enum** en C). Le signal **etat** sera synthétisé comme le nombre nécessaire de registres pour pouvoir encoder toutes les valeurs possibles de **etat**, déclarées par **etat_type**. À la manière d'un **enum**, l'état **e_1** est encodé par "00", **e_2** par "01", **e_3** par "10" et **e_0** par "11". Cela correspond à l'ordre dans lequel ils sont déclarés.

Simulation

Pour commencer, simulez le module. Pour rappel, mettez le **feu_tb.vhd** en *top level* dans les « Simulation sources ». Puis, dans le Flow Navigator, faites « Run Simulation » et observez le chronogramme. Vous devriez constater que le banc d'essai stimule le bouton d'entrée un tick d'horloge sur deux. Cela permet de tester à la fois les transitions, et la persistance des états. Vous pouvez aussi observer que les deux sorties du module suivent un cycle de valeurs en décalé. Prenez le temps de comprendre le banc d'essai fourni.

Synthèse/implémentation/*bitstream*

Exécutez ensuite l'ensemble du flot matériel permettant la génération du *bitstream*. Pour rappel, mettez le **top.vhd** en *top level* dans les « Design sources ». Puis, dans le Flow Navigator, faites « Generate Bitstream ».

Programmation/validation

Avec votre PYNQ-Z2 branchée et bootée, programmez la carte en cliquant sur « Program Device » dans le « Hardware Manager », et interagissez avec le module pour vérifier son bon fonctionnement. Notez qu'il n'y a pas besoin de *notebook* Jupyter dans ce laboratoire puisque seules les entrées/sorties de la carte seront utilisées (LEDs et boutons).

Tiens tiens tiens... Vous devriez constater que le module ne se comporte pas comme prévu. Pourtant :

- la simulation indique que les LEDs passent bien de « vert/blanc » à « blanc/rouge » au premier appui, de « blanc/rouge » à « rouge/vert » au second, de « rouge/vert » à « vert/blanc » au troisième, ...
- et un module *debouncer* (*monopulseur.vhd*) est déjà en place et supprime les effets de rebond du bouton (voir [cette vidéo](#) pour plus de renseignements).

Alors, comment se fait-il que l'implémentation sur la carte ne semble pas fonctionnelle ? Implémentez **un process dans votre module feu.vhd pour résoudre le problème, et vérifiez-le sur la carte**. N'hésitez pas à utiliser autant de signaux intermédiaires que nécessaire.

! Évaluation

- **Pour l'entrevue/démonstration**, soyez capable d'expliquer brièvement le problème rencontré et la manière dont vous l'avez résolu.

Partie 2 : Feu de circulation

Spécifications

Dans cette partie, il vous est demandé d'implémenter une machine à états qui respecte les spécifications suivantes pour mettre en place un feu de circulation. **Prenez le temps de bien les lire en intégralité.**

- Les **deux feux de circulation** considérés sont les suivants :
 - le **Feu 0** est affiché sur la **LED RGB de droite** (LD4) et associé au **Bouton 0** (BTN0, le plus à droite),
 - le **Feu 1** est affiché sur la **LED RGB de gauche** (LD5) et associé au **Bouton 1** (BTN1, à gauche du Bouton 0).
- **Toutes les transitions** ont lieu sur le **front montant** de l'horloge.
- Pour la sécurité des usagers et la fluidité du trafic :
 - si un feu est **vert**, l'autre est forcément **rouge**,
 - si un feu est **rouge**, l'autre est forcément **vert ou jaune**.
- Dans l'**état initial**, les deux feux doivent être **éteints**.

- Lorsque le système est dans son **état initial**, l'appui sur un bouton provoque l'**allumage direct du feu correspondant en vert**.
- Lorsque le système est dans un **état de fonctionnement** (différent de l'état initial), l'appui sur un bouton fait office d'**appel piéton**. Concrètement, si le Bouton 0 (resp. 1) est appuyé :
 - si le Feu 0 (resp. 1) était vert, il doit passer **au jaune pendant deux secondes, puis au rouge**,
 - si le Feu 0 (resp. 1) n'était pas vert, l'appui est sans effet.
- Le **reset** est **asynchrone**, relié à un **appui simultané des deux boutons** et provoque un **retour à l'état initial**.

Diagramme d'états

Représentez la machine que vous allez implémenter sous forme d'un **diagramme d'états**. Le diagramme doit comporter tous les états utilisés, toutes les transitions en fonction des entrées, et les valeurs des sorties pour chaque état. Vous êtes libres d'utiliser autant d'états que vous désirez, sachant que la correction implémente ces spécifications en **5 états** (dont l'état initial).

! Évaluation

- **Pour l'entrevue/démonstration**, soyez prêt à montrer et décrire votre diagramme d'états : il s'agit d'une étape importante de la phase de conception.

Partie 3 : Implémentation VHDL et simulation

Préambule : Gestion des *timings*

Comme précisé dans les spécifications, le feu jaune doit durer deux secondes, sachant que l'horloge qui régit le fonctionnement du FPGA est cadencée à 125 MHz. Un generic nommé `YELLOW_DURATION_CYCLES` vous est fourni dans le `feu.vhd`, et contient le **nombre de cycles d'horloge** que doit durer le passage au feu jaune. Votre **gestion de ce délai doit fonctionner pour n'importe quelle valeur de `YELLOW_DURATION_CYCLES`, et doit donc l'utiliser**.

Notez que ce generic est déjà assigné :

- à 250×10^6 dans le `feu.vhd`, ce qui est parfait pour que ce passage dure deux secondes lors du test sur la PYNQ-Z2,
- et à 8 dans le banc d'essai `feu_tb.vhd`, ce qui est parfait pour visualiser ce passage sur les chronogrammes en simulation.

Ne touchez donc pas aux valeurs déjà assignées dans les différents fichiers pour ce generic. La Figure 2 (page suivante) vous montre le chronogramme que vous devrez observer en simulation, avec donc `YELLOW_DURATION_CYCLES = 8`.

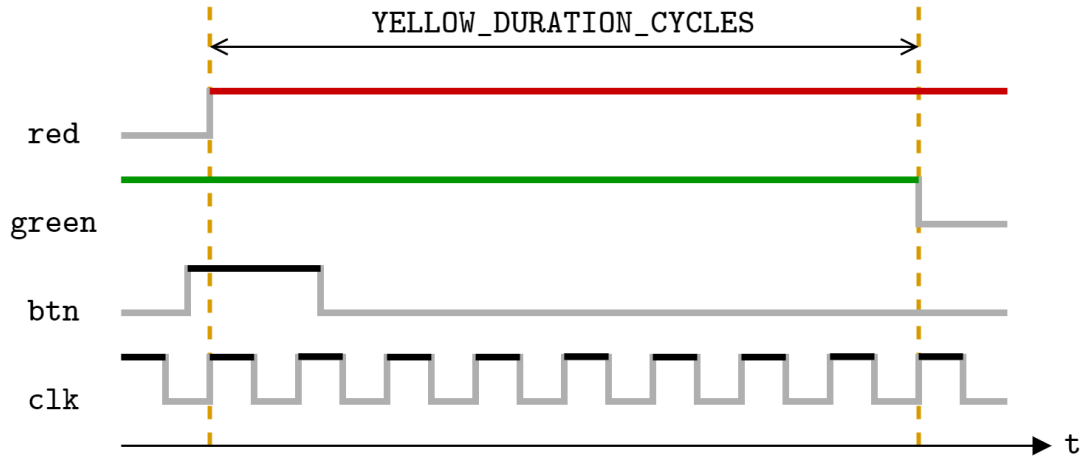


Fig. 2. – Chronogramme du passage au feu jaune pour `YELLOW_DURATION_CYCLES = 8`

Implémentation VHDL

Modifiez le fichier `feu.vhd` pour respecter les spécifications décrites dans la **Partie 2**. Pour que votre module fonctionne sur la carte, **n’effacez pas le process que vous avez codé dans la Partie 1**. Il est fortement conseillé de produire un **code clair, lisible, et commenté**, afin de pouvoir plus facilement s’y retrouver durant l’entrevue/démonstration.

! Évaluation

- **Pour la remise de code**, *pushez* votre fichier `feu.vhd` fonctionnel sur votre dépôt GitHub.
- **Pour l’entrevue/démonstration**, soyez capable de présenter votre architecture finale et d’expliquer en quoi elle permet de respecter les spécifications.

Simulation

Validez votre module en écrivant dans `feu_tb.vhd` un **banc d’essai permettant de tester toutes les transitions possibles**.

! Évaluation

- **Pour la remise de code**, *pushez* votre fichier `feu_tb.vhd` complet sur votre dépôt GitHub.
- **Pour l’entrevue/démonstration**, soyez capable de décrire votre banc d’essai et de justifier comment les résultats de simulation vous permettent de conclure au bon fonctionnement du modèle.

Partie 4 : Synthèse et implémentation sur la carte

Générez le *bitstream*, programmez la carte, et vérifiez le bon fonctionnement de votre feu de circulation.

! Évaluation

- **Pour la remise de code**, *pushez* votre *bitstream* `top.bit` fonctionnel sur votre dépôt GitHub. Il est placé dans `<project_dir>/<project_name>.runs/impl_1/`.
- **Pour l’entrevue/démonstration**, soyez prêt à faire une démonstration sur la carte que toutes les spécifications ont été rencontrées.

Partie 5 [Défi] : Feu clignotant

! Mise en garde

Compléter correctement les parties précédentes peut donner une note de 17/20 (85%), ce qui peut normalement être interprété comme un A. Cette dernière partie propose un défi supplémentaire pour les personnes qui souhaitent s’investir davantage dans le cours INF3500. Il n’est pas nécessaire de la compléter pour réussir le cours, ni pour obtenir une bonne note.

Dans un souci de réalisme à toute épreuve, on vous demande d’ajouter la fonctionnalité suivante au feu que vous avez implémenté : faire clignoter le feu jaune avant son passage au rouge. Concrètement, lorsque le Bouton 0 (resp. 1) est appuyé, le Feu 0 (resp. 1) doit :

- passer au **jaune fixe sur la première seconde**,
- **clignoter** en jaune **sur la deuxième seconde**.

En combinaison du `YELLOW_DURATION_CYCLES` de la **Partie 3**, deux constantes vous sont fournies pour paramétrer ce clignotement :

- `BLINK_START_CYCLES` correspond au nombre de cycles avant que le clignotement ne démarre, soit la moitié de la durée du feu jaune,
- `BLINK_TOGGLE_CYCLES` correspond au nombre de cycles avant de basculer (*toggle*) le clignotement. On vous propose une valeur de 1/8 de la durée du feu jaune, permettant d’afficher deux clignotements.

Ne modifiez pas les valeurs assignées, car elles s’adaptent comme faut aux différentes valeurs de `YELLOW_DURATION_CYCLES`. La Figure 3 (page suivante) vous montre le chronogramme que devrez observer en simulation, avec le même `YELLOW_DURATION_CYCLES = 8`.

Votre gestion du clignotement devra utiliser les deux constantes sus-citées, toujours dans l’optique de concevoir une architecture générique.

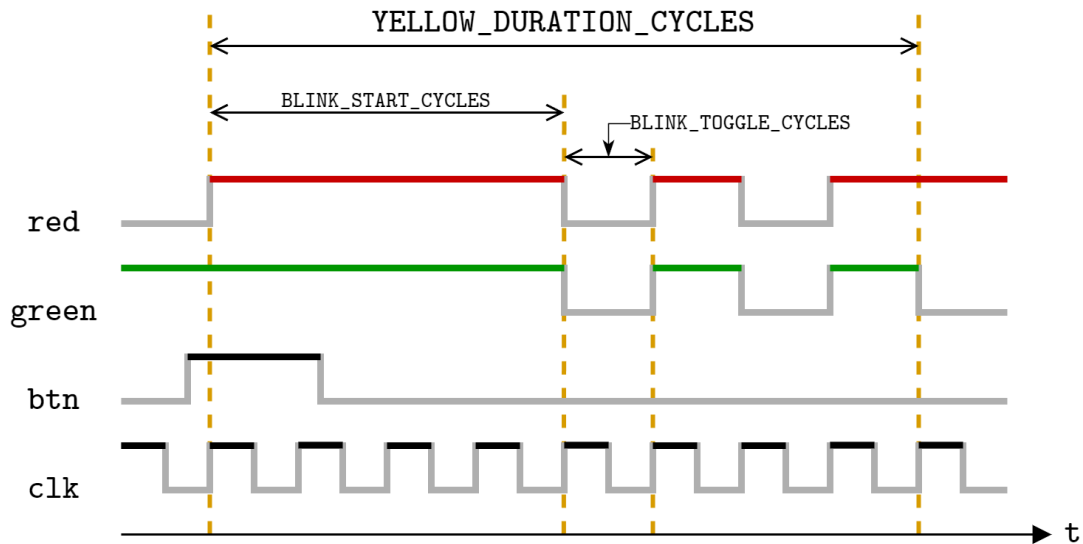


Fig. 3. – Chronogramme du passage au feu jaune avec clignotements pour $\text{YELLOW_DURATION_CYCLES} = 8$

⚠ Avant de continuer...

Assurez-vous d'avoir déjà *push* vos fichiers `feu.vhd`, `feu_tb.vhd` et `top.bit` fonctionnels jusque-là. Il vaut mieux prendre les devants au cas où les choses tourneraient mal.

Modifiez vos fichiers `feu.vhd` et `feu_tb.vhd` pour implémenter **et vérifier** cette nouvelle fonctionnalité.

! Évaluation

- **Pour la remise de code**, *push*ez vos fichiers `feu.vhd`, `feu_tb.vhd` et `top.bit` modifiés.
- **Pour l'entrevue/démonstration**, soyez capable de décrire comment vous avez ajouté et validé cette fonctionnalité, avec en point d'orgue la démonstration du clignotement sur la carte !